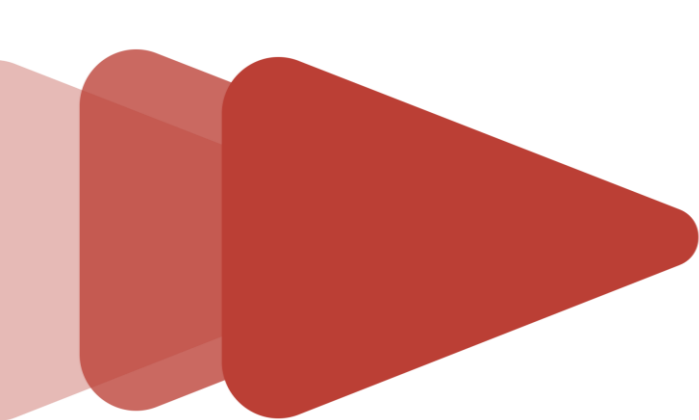




Estrategias de despliegue y herramientas para las fases Monitoreo y Operación.

30 de mayo 2024

Contenido

1.	Introducción.....	2
2.	Ansible en el proceso de CI/CD	2
	Arquitectura y Componentes.....	3
	Ventajas del uso de Ansible en CI/CD	3
	Integración de Ansible en el Proceso de CI/CD.....	3
	Automatización de la implementación con Ansible	4
3.	Estrategias de Despliegue	4
	Blue-Green Deployment	5
	Canary Deployment.....	5
	Rolling Deployment	6
	A/B Testing	7
	Shadow Deployment	7
	Feature Toggles	8
	Deployment Rings.....	8
	Rollback Automático	9
	Factores en las Estrategias de Despliegue	9
	Estrategias de Despliegues para Arquitecturas Monolíticas y Resilientes	12
	Desafíos y Ventajas en Estrategias de Despliegue en Arquitecturas Monolíticas y Resilientes	13
	Comparativa de las Estrategias de Despliegue vs Factores a Considerar	15
4.	Análisis de Casos de Uso y Mejores Prácticas para Estrategias de Despliegue en Arquitecturas Resilientes	16
5.	Herramientas para las fases Monitor y Operate.....	17
6.	Conclusiones	21
7.	Recomendaciones.....	21
8.	Glosario	22
9.	Bibliografía.....	22

1. Introducción

Contexto del informe

Objetivo general

El objetivo es analizar las estrategias de despliegue de software existentes para el uso en arquitecturas monolíticas y resilientes.

Objetivos específicos:

Automatización del Proceso de Despliegue:

- Analizar el uso de Ansible en los procesos de CI/CD orientados al uso en arquitecturas resilientes.
- Identificar y analizar las características principales, ventajas y desventajas de las estrategias de despliegue.
- Identificar de las estrategias de despliegue analizadas cuales permitirán garantizar una actualización del servicio sin generar indisponibilidad.

2. Ansible en el proceso de CI/CD

En la actualidad, el desarrollo de software se enfrenta a demandas crecientes en términos de velocidad, calidad y eficiencia sin descuidar la parte fundamental de la seguridad. La adopción de las prácticas de CI/CD se ha convertido en un elemento estratégico para satisfacer las demandas actuales en TI, enfocándose en mejorar los tiempos de entrega de mejoras en el desarrollo en ambientes críticos con una de las premisas más importantes que sea un software confiable. En este contexto, Ansible emerge como una solución para los diversos actores en el manejo de infraestructura TI y sobre todo para los administradores de infraestructura quienes obtienen más ventaja para mejorar sus procesos y automatizar diferentes tareas relacionadas a sus funciones como el aprovisionamiento, configuración y despliegue de infraestructura y aplicaciones. Ansible se caracteriza por su simplicidad, flexibilidad y capacidad de automatización lo que la convierte en una de las herramientas neurálgicas para integrarse con los otros procesos de CI/CD.

La Integración continua (CI) es el proceso de incorporar los cambios de código a un repositorio compartido de código fuente de forma automática y periódica.

El éxito de la integración continua (CI) requiere una validación completa de las modificaciones del desarrollador. Después de la incorporación de cambios, se realiza la compilación automática y pruebas automatizadas, abarcando desde clases individuales hasta módulos completos de la aplicación. La CI facilita la rápida resolución de errores detectados por pruebas automáticas, asegurando la compatibilidad entre el código nuevo y existente.

La Distribución Continua (CD) es el proceso en el que se integran, prueban y distribuyen los cambios de código.

Hay que diferenciar que la entrega continua no llega al despliegue automático en producción, mientras

que la Integración continua libera automáticamente las actualizaciones en el entorno de producción.

En conjunto, estas prácticas conectadas a menudo se denominan integración y distribución continua "CI/CD" y cuentan con el respaldo de equipos de desarrollo y operaciones que trabajan juntos de manera ágil con un enfoque de DevOps o de ingeniería de confiabilidad del sitio (SRE).

Ansible utiliza un enfoque basado en YAML para definir la infraestructura como código, lo que permite a los equipos de desarrollo y operaciones automatizar procesos de manera eficiente y reproducible.

Arquitectura y Componentes

La arquitectura de Ansible se basa en un enfoque agenteless, lo que significa que no requiere la instalación de agentes en los sistemas a gestionar. La comunicación se establece por SSH con todos los equipos listados en un inventario. Los componentes principales de Ansible incluyen:

- **Control Node:** Es el nodo desde el cual se ejecutan los comandos de Ansible. Aquí se encuentra instalado Ansible y desde aquí se coordinan las acciones hacia los nodos gestionados.
- **Managed Nodes:** Son los sistemas remotos que son gestionados por Ansible. Estos nodos pueden ser servidores, dispositivos de red o cualquier otro tipo de máquina accesible por SSH.
- **Inventario (Inventory):** Es un archivo que contiene información sobre los nodos gestionados por Ansible. Puede ser estático o dinámico y permite organizar y clasificar los nodos en grupos para facilitar su gestión.
- **Módulos (Modules):** Son pequeños programas escritos en Python que realizan tareas específicas en los nodos gestionados. Ansible utiliza módulos para realizar acciones como instalar paquetes, reiniciar servicios o copiar archivos, dependiendo de la evolución en el uso de Ansible, las organizaciones de TI pueden crear sus propios módulos lo que simplificará más aún los procesos automatizados.
- **Playbooks:** Son archivos YAML que contienen una serie de tareas que Ansible debe ejecutar en los nodos gestionados. Los playbooks permiten definir de manera declarativa el estado deseado del sistema y las acciones necesarias para alcanzar ese estado.

Ventajas del uso de Ansible en CI/CD

Automatización: Ansible simplifica la automatización de tareas repetitivas y propensas a errores, lo que aumenta la eficiencia y reduce el tiempo dedicado a la administración de sistemas.

- **Consistencia:** Al definir la infraestructura como código, Ansible garantiza que los entornos de desarrollo, pruebas y producción sean consistentes y reproducibles.
- **Flexibilidad:** Ansible es altamente flexible y puede integrarse fácilmente con otras herramientas y sistemas existentes en el entorno de desarrollo.
- **Escalabilidad:** Ansible es adecuado para proyectos de cualquier tamaño, desde pequeños equipos hasta grandes empresas, gracias a su capacidad para gestionar grandes flotas de servidores de manera eficiente.
- **Comunidad activa:** Ansible cuenta con una comunidad activa de desarrolladores y usuarios que contribuyen con módulos, playbooks y recursos de aprendizaje, lo que facilita la curva de aprendizaje, así como su uso.

Integración de Ansible en el Proceso de CI/CD

El proceso de Integración Continua y Despliegue Continuo (CI/CD) es una práctica de desarrollo de software que consiste en integrar y desplegar cambios en el código de manera continua y automatizada. Este proceso se divide:

- **Integración Continua (CI):** En esta etapa, los desarrolladores integran sus cambios de código en un repositorio compartido varias veces al día. Cada vez que se realiza una integración, se ejecutan pruebas automáticas para verificar la calidad del código.
- **Despliegue Continuo (CD):** Después de que los cambios se han integrado correctamente, el código se despliega automáticamente en un entorno de pruebas o producción. Este despliegue automático garantiza que las nuevas características y correcciones de errores lleguen rápidamente a los usuarios finales.

Papel de Ansible en el proceso CI/CD

Ansible desempeña un papel crucial en el proceso de CI/CD al automatizar tareas de aprovisionamiento, configuración y despliegue de infraestructura y aplicaciones. Algunas de las formas en que Ansible contribuye al proceso son:

- **Automatización de la Configuración del Entorno:** Ansible permite definir la configuración del entorno de desarrollo, pruebas y producción como código, lo que garantiza que todos los entornos sean consistentes y reproducibles.
- **Despliegue de Aplicaciones:** Ansible puede automatizar el despliegue de aplicaciones en entornos de pruebas y producción, asegurando que las nuevas versiones se implementen de manera rápida y confiable.
- **Ejecución de Pruebas y Validaciones:** Ansible puede integrarse con herramientas de prueba y validación para ejecutar pruebas automáticas en los sistemas desplegados, garantizando la calidad del código antes de su implementación.
- **Gestión de la Infraestructura:** Ansible puede gestionar la infraestructura subyacente necesaria para ejecutar las aplicaciones, incluyendo la creación y configuración de servidores, redes y servicios.

Automatización de la implementación con Ansible

La automatización de la implementación con Ansible implica la creación de playbooks que describen los pasos necesarios para implementar una aplicación o actualizar un entorno. Estos playbooks pueden incluir tareas como la instalación de dependencias, la configuración de servicios y la copia de archivos. Al automatizar estos procesos con Ansible, se reduce el riesgo de errores humanos y se acelera el tiempo de implementación. Además, la infraestructura y las aplicaciones se gestionan de manera consistente en todos los entornos, desde el desarrollo hasta la producción.

3. Estrategias de Despliegue

Las estrategias de despliegue son mecanismos y prácticas con la capacidad de actualizar o cambiar una aplicación o configuración sin generar periodos de inactividad. Permiten a los equipos de DevOps definir cómo se desplegarán las aplicaciones en el entorno de producción. La correcta elección de la estrategia de despliegue permitirá a los administradores de infraestructura minimizar el riesgo, impacto de desplegar una nueva versión para los usuarios finales, así también, permitirán dimensionar la infraestructura necesaria para implementar dicha estrategia.

Las estrategias de despliegue consideran los procesos de integración continua, debido a que estos ayudarán en el manejo de la seguridad y calidad del código a desplegar. Asimismo, este recurso permitirá la identificación temprana de vulnerabilidades, el testing del software o la detección de cuellos de botella, entre otras opciones.

Una característica fundamental de estas estrategias de despliegue es su capacidad para revisar el código y los cambios implementados. Este aspecto es crucial para gestionar eficazmente las nuevas características integradas y garantizar un rendimiento óptimo en el sistema. Además, estas estrategias están diseñadas para automatizar procesos y supervisar el funcionamiento del servicio desplegado de

manera eficiente.

Blue-Green Deployment

Consiste en tener dos entornos idénticos, uno de producción (blue) y otro de pruebas (green). La versión actual de la aplicación se aprovisiona en el entorno activo, mientras que la nueva versión se despliega en el entorno en espera. Se dirige el tráfico hacia el entorno de pruebas para realizar el despliegue, y una vez validado, se redirige el tráfico hacia el entorno de producción. El despliegue en el entorno de espera no tiene impacto en el entorno activo ni en el tráfico de los usuarios.

Esta técnica permite al equipo DevOps evaluar la nueva versión de la aplicación con respecto al tráfico de usuario real. Pueden comparar las dos versiones de la aplicación en paralelo antes de desplegar la nueva versión en una base de usuarios más grande. También ofrece mitigación de riesgos, ya que la nueva versión solo está activada para un pequeño subconjunto de usuarios. Estos usuarios pueden volver fácilmente a la versión anterior si surgen problemas.

Un despliegue de Blue Green ofrece estas ventajas:

- Permite despliegues rápidos y sin riesgos.
- Ofrece un mecanismo de rollback eficaz y sencillo.
- Es una forma eficaz de organizar pruebas de software A/B.
- Requiere poco o ningún tiempo de inactividad, ya que la producción siempre es servida por uno de los entornos activos (controlado por un equilibrador de carga).

Sin embargo, debe tener en cuenta estos inconvenientes:

- Ejecutar entornos idénticos es caro y el mantenimiento requiere muchos recursos.
- Al gestionar una versión entre dos entornos idénticos, debe supervisar de cerca ambos entornos.
- La gestión de dependencias de base de datos entre despliegues puede ser complicada.

La principal ventaja de esta estrategia es que ofrece un tiempo de inactividad prácticamente nulo y la capacidad de revertir instantáneamente a una versión anterior en caso de problemas. Si surgen problemas con la nueva versión, el tráfico puede volver de inmediato a la versión estable anterior.

Por ejemplo, Suponiendo que se tiene un sistema de gestión de contenido en la nube que se ejecuta en dos entornos separados: el entorno "Azul" y el entorno "Verde". Inicialmente, todo el tráfico de usuarios se dirige al entorno "Azul", que es el entorno activo y contiene la versión actual del sistema. Se decide lanzar una actualización importante para el sistema en el entorno "Verde". Esta actualización incluye mejoras significativas en la interfaz de usuario y en la velocidad de procesamiento. Después de implementar la actualización en el entorno "Verde", se debe comenzar a redirigir gradualmente una pequeña parte del tráfico de usuarios hacia este entorno. Esta parte del tráfico representa a un grupo seleccionado de usuarios que han optado por probar nuevas características. Mientras el grupo de usuarios está utilizando el entorno "Verde" y proporcionando comentarios valiosos sobre la nueva actualización, el resto de los usuarios sigue accediendo al entorno "Azul". Una vez que se haya verificado que la actualización en el entorno "Verde" funciona correctamente y no hay problemas significativos, se decide cambiar todo el tráfico de usuarios al entorno "Verde". Ahora, el entorno "Verde" se convierte en el entorno activo y principal para tu sistema. Ahora que la actualización se ha implementado con éxito y todos los usuarios están utilizando el entorno "Verde", se puede realizar cualquier mantenimiento necesario en el entorno "Azul" o prepararlo para la próxima actualización sin afectar a la experiencia del usuario.

Canary Deployment

Se implementa el nuevo código en un pequeño subconjunto de servidores o usuarios en un entorno sin tráfico, y se monitorea su comportamiento. Si todo funciona correctamente, se despliega gradualmente en el resto del entorno. Si hay problemas, se puede revertir fácilmente.

Permite probar cambios en un entorno de producción real con un riesgo mínimo al limitar el impacto inicialmente a un grupo reducido de usuarios o servidores.

Los despliegues canarios ofrecen estas ventajas:

- Puede probar dos versiones de aplicaciones en paralelo con usuarios reales.
- Sin tiempo de inactividad para las nuevas versiones.
- La reversión a una versión anterior es muy fácil y conlleva el menor riesgo.

Sin embargo, debe tener en cuenta estos inconvenientes:

- La prueba y validación de una nueva versión a escala puede ser compleja.
- La recuperación de los comentarios de las pruebas de usuario en una nueva versión requiere mucho tiempo.

Esta técnica permite al equipo DevOps evaluar la nueva versión de la aplicación con respecto al tráfico de usuario real. Pueden comparar las dos versiones de la aplicación en paralelo antes de desplegar la nueva versión en una base de usuarios más grande. También ofrece mitigación de riesgos, ya que la nueva versión solo está activada para un pequeño subconjunto de usuarios. Estos usuarios pueden volver fácilmente a la versión anterior si surgen problemas.

Por ejemplo, Una aplicación de redes sociales implementa una nueva función de publicación de historias utilizando Canary Deployment. Inicialmente, solo un pequeño porcentaje de usuarios tienen acceso a la nueva función, mientras el resto sigue usando la versión anterior. Se monitorea el rendimiento y se recopilan comentarios de los usuarios canarios. Gradualmente, se aumenta el porcentaje de usuarios con acceso a la nueva función hasta que todos la tienen. Si se detectan problemas, se puede revertir fácilmente el despliegue sin afectar a todos los usuarios. Este enfoque permite probar y validar la nueva función antes de su lanzamiento completo, minimizando riesgos y garantizando una transición suave.

Rolling Deployment

Consiste en ir actualizando los servidores o instancias del sistema de forma gradual en diferentes instancias del ambiente de producción, uno tras otro, asegurando que siempre haya una versión del sistema disponible y funcional. Minimiza el impacto del despliegue al distribuir la carga entre múltiples instancias y permitir una actualización controlada y escalonada. Esto significa que el tráfico del usuario se sigue dirigiendo a las instancias que aún no han sido actualizadas.

Los despliegues escalonados ofrecen estas ventajas:

- Disponibilidad continua del servicio: Los usuarios pueden seguir accediendo a la aplicación sin interrupciones mientras se actualizan las instancias.
- Riesgo controlado: Si algo sale mal con la nueva versión, solo afecta a una parte del tráfico de usuarios, lo que limita el impacto.
- Capacidad de retroceso fácil: Si se detecta un problema, se puede detener el despliegue y revertir rápidamente a la versión anterior.

Sin embargo, es importante considerar estos inconvenientes:

- Tiempo de despliegue prolongado: Debido a la naturaleza gradual del proceso, el despliegue puede llevar más tiempo que otros métodos.
- Complejidad de la coordinación: Coordinar la actualización de múltiples instancias puede ser más complicado que un despliegue completo.

Esta técnica permite al equipo de DevOps implementar cambios de manera controlada y sin interrupciones en el servicio. Al actualizar gradualmente las instancias, pueden detectar y resolver problemas antes de que afecten a todos los usuarios. Además, la capacidad de revertir fácilmente a la versión anterior proporciona una capa adicional de seguridad en caso de problemas inesperados.

Por ejemplo, si tenemos un clúster de servidores que ejecutan nuestra aplicación, en lugar de actualizar todos a la vez, se actualizan uno tras otro, asegurando que siempre haya una versión funcional del sistema disponible. Esto significa que el tráfico del usuario se sigue dirigiendo a las instancias que aún no han sido actualizadas.

A/B Testing

Se lanzan dos versiones diferentes del mismo servicio o aplicación y se dirige el tráfico de manera aleatoria entre ambas versiones. Se recopilan datos y se analiza el rendimiento para tomar decisiones basadas en evidencia.

Permite comparar el rendimiento y la aceptación de dos o más versiones de una aplicación en un entorno de producción real antes de implementarla completamente.

Los despliegues de A/B Testing ofrecen estas ventajas:

- Comparación directa de versiones: Permite evaluar directamente el impacto de los cambios en el comportamiento de los usuarios al presentarles diferentes versiones de la aplicación.
- Validación basada en datos: La recopilación de datos sobre el rendimiento de ambas versiones proporciona información objetiva para tomar decisiones sobre qué versión implementar completamente.
- Mitigación de riesgos: Al limitar el impacto inicial a una parte de los usuarios, se reduce el riesgo de problemas graves que afecten a todos los usuarios.

Sin embargo, es importante considerar estos inconvenientes:

- Necesidad de un tamaño de muestra adecuado: Para obtener resultados significativos, se requiere un número suficiente de usuarios en cada grupo de prueba.
- Posible confusión para los usuarios: Al presentar versiones diferentes a diferentes usuarios, puede generar confusión o insatisfacción si no se comunica claramente el propósito del A/B Testing.

Esta técnica permite al equipo de DevOps probar cambios en un entorno de producción real antes de implementarlos completamente. Al recopilar datos sobre el rendimiento y la aceptación de ambas versiones, pueden tomar decisiones informadas sobre qué características o diseños son más efectivos para sus usuarios.

Por ejemplo, si se está lanzando una nueva funcionalidad o diseño, una parte de los usuarios verá la versión A (la versión existente) y otra parte verá la versión B (la nueva versión). Se recopilan datos sobre el comportamiento y las preferencias de los usuarios en ambas versiones para evaluar su rendimiento.

Shadow Deployment

Se implementa una nueva versión de la aplicación en paralelo con la versión existente, pero no se enruta tráfico de usuarios hacia ella. En su lugar, se utilizan para recopilar datos y realizar pruebas de rendimiento y funcionalidad. Permite probar nuevas versiones en un entorno de producción real sin afectar a los usuarios finales, ya que el tráfico real sigue dirigido hacia la versión existente.

Los despliegues en Shadow ofrecen estas ventajas:

- Pruebas sin impacto en los usuarios: Permite probar la nueva versión en un entorno de producción real sin afectar la experiencia de los usuarios finales, ya que el tráfico sigue dirigido hacia la versión existente.
- Recopilación de datos: Al recopilar datos y realizar pruebas en la nueva versión mientras está en la sombra, se puede evaluar su rendimiento y funcionalidad antes de implementarla completamente.
- Mitigación de riesgos: Si surgen problemas con la nueva versión, no afectan a los usuarios, ya que el tráfico sigue dirigido hacia la versión estable existente. Esto proporciona una capa adicional de seguridad durante el proceso de implementación.

Sin embargo, es importante considerar estos inconvenientes:

- Complejidad adicional: La implementación de una versión en paralelo con la existente puede añadir complejidad al entorno de producción y al proceso de despliegue.

- Sincronización de datos y funcionalidades: Es importante asegurarse de que la nueva versión en la sombra esté sincronizada con la versión existente en términos de datos y funcionalidades para obtener resultados de prueba precisos.

Esta técnica permite al equipo de DevOps probar y validar una nueva versión en un entorno de producción real antes de implementarla completamente, minimizando así el riesgo de problemas o fallos para los usuarios finales.

Por ejemplo, si estamos lanzando una actualización importante, la nueva versión se implementa en un entorno de producción real, pero el tráfico de los usuarios continúa dirigiéndose hacia la versión existente. En lugar de servir a los usuarios, la nueva versión se utiliza para recopilar datos y realizar pruebas de rendimiento y funcionalidad.

Feature Toggles

Se implementan interruptores de características en el código que permiten activar o desactivar funcionalidades específicas sin necesidad de una nueva versión del software.

Permite habilitar o deshabilitar funcionalidades de manera independiente, lo que facilita la realización de pruebas, la personalización del producto y la gestión de lanzamientos.

Los Feature Toggles ofrecen estas ventajas:

- Control granular de las funcionalidades: Permite habilitar o deshabilitar funcionalidades de manera independiente, lo que facilita la realización de pruebas y la personalización del producto.
- Implementación gradual: Las nuevas funcionalidades pueden implementarse gradualmente y activarse cuando estén listas, en lugar de tener que esperar a una versión completa del software.
- Mitigación de riesgos: Si una nueva funcionalidad causa problemas inesperados, se puede desactivar fácilmente, evitando así impactos negativos en la experiencia del usuario.

Sin embargo, es importante considerar estos inconvenientes:

- Complejidad técnica: La gestión de múltiples interruptores de características puede añadir complejidad al código y al proceso de desarrollo.
- Posible degradación del rendimiento: El uso excesivo de Feature Toggles puede ralentizar el rendimiento de la aplicación si no se gestionan adecuadamente.

Esta técnica permite al equipo de DevOps tener un mayor control sobre las funcionalidades que se implementan y cómo se entregan a los usuarios finales. Al activar y desactivar funcionalidades de manera selectiva, pueden realizar pruebas y obtener retroalimentación de los usuarios antes de implementar completamente las nuevas características.

Por ejemplo, si se está desarrollando una nueva funcionalidad compleja, en lugar de lanzarla completamente una vez que esté lista, se puede implementar parcialmente y activarla solo para un grupo seleccionado de usuarios o en un entorno específico.

Deployment Rings

Consiste en dividir el entorno de producción en anillos o grupos, donde se despliegan primero las nuevas versiones en un grupo limitado antes de propagarse al resto de los anillos. Proporciona un enfoque escalonado para los despliegues, lo que permite identificar y solucionar problemas en un grupo reducido antes de expandirse a toda la infraestructura.

Los Deployment Rings ofrecen estas ventajas:

- Enfoque escalonado: Proporciona un enfoque escalonado para los despliegues, lo que permite identificar y solucionar problemas en un grupo reducido antes de expandirse a toda la infraestructura.

- Control de riesgos: Limita el impacto de un despliegue fallido al restringir inicialmente la nueva versión a un grupo seleccionado de servidores o regiones.
- Flexibilidad: Permite ajustar la velocidad y el alcance del despliegue en función de las necesidades y la capacidad del equipo de DevOps.

Sin embargo, es importante considerar estos inconvenientes:

- Complejidad de gestión: Administrar múltiples anillos de despliegue puede añadir complejidad al proceso de implementación y requerir una coordinación cuidadosa.
- Posible fragmentación del entorno: Si no se gestionan adecuadamente, los diferentes anillos de despliegue pueden derivar en una fragmentación del entorno de producción.

Esta técnica permite al equipo de DevOps realizar despliegues de manera controlada y gradual, lo que facilita la identificación y resolución de problemas antes de que afecten a toda la infraestructura. Al implementar primero en un grupo limitado y luego expandirse, se reduce el riesgo y se garantiza una transición suave hacia la nueva versión.

Por ejemplo, si tenemos un entorno de producción con múltiples servidores o regiones, podemos comenzar desplegando una nueva versión en un anillo de servidores seleccionado antes de extenderla gradualmente a los anillos restantes.

Rollback Automático

Se establecen mecanismos automáticos para revertir un despliegue si se detectan problemas, como errores en el sistema o una degradación del rendimiento. Minimiza el impacto de un despliegue fallido al restaurar automáticamente la versión anterior del software en caso de problemas, sin intervención manual.

El Rollback Automático ofrece estas ventajas:

- Minimización del impacto: Permite mitigar rápidamente los efectos negativos de un despliegue fallido o problemático al revertir automáticamente a una versión anterior.
- Reducción del tiempo de inactividad: Al automatizar el proceso de rollback, se minimiza el tiempo que el sistema permanece en un estado no deseado, lo que ayuda a mantener la disponibilidad del servicio.
- Simplicidad en la gestión de incidentes: Facilita la gestión de incidentes al eliminar la necesidad de intervención manual para realizar un rollback, lo que permite una respuesta más rápida y eficiente.

Sin embargo, es importante considerar estos inconvenientes:

- Posibilidad de rollback incorrecto: Si no se implementa correctamente, el rollback automático puede revertir a una versión anterior sin una comprensión completa de la causa raíz del problema, lo que puede no resolver el problema subyacente.
- Necesidad de una monitorización sólida: Es crucial contar con sistemas de monitorización robustos y alertas configuradas adecuadamente para detectar problemas y desencadenar el rollback automático de manera oportuna y precisa.

Esta técnica permite al equipo de DevOps mantener la estabilidad y la disponibilidad del servicio al revertir automáticamente despliegues problemáticos. Al reducir el tiempo necesario para responder a problemas en producción, se mejora la experiencia del usuario y se minimizan los impactos negativos en el negocio.

Por ejemplo, si se monitorea el sistema y se detecta un aumento significativo en los errores o una degradación del rendimiento después de un despliegue, el sistema puede activar automáticamente el proceso de rollback para restaurar la versión anterior.

Factores en las Estrategias de Despliegue

Existen varios factores en las estrategias de despliegue que influyen en la efectividad y el éxito de la

implementación de cambios en infraestructuras con arquitecturas monolíticas o resilientes, a continuación, se describen los principales factores que deben considerar para realizar la definición del uso de una estrategia de despliegue:

- Impacto en la Disponibilidad
- Riesgo de Tiempo de Inactividad
- Complejidad de Implementación
- Capacidad de Revertir Cambios
- Velocidad de Implementación
- Impacto en la Experiencia del Usuario
- Necesidad de Infraestructura Adicional
- Tolerancia a Fallos
- Complejidad de Gestión del Estado
- Escalabilidad

Impacto en la Disponibilidad: La disponibilidad del servicio es crucial para garantizar que los usuarios puedan acceder al sistema en todo momento. Las estrategias de despliegue que minimizan el impacto en la disponibilidad ayudan a mantener la continuidad del servicio y la satisfacción del usuario.

Riesgo de Tiempo de Inactividad: El tiempo de inactividad puede causar interrupciones en el servicio, pérdida de ingresos y dañar la reputación de la marca. Reducir el riesgo de tiempo de inactividad es fundamental para mantener la estabilidad del sistema durante las actualizaciones.

Complejidad de Implementación: La complejidad de implementación afecta la facilidad con la que se pueden realizar cambios en el sistema. Estrategias de despliegue más simples pueden ser más rápidas de implementar y menos propensas a errores, mientras que estrategias más complejas pueden requerir más recursos y planificación.

Capacidad de Revertir Cambios: La capacidad de revertir cambios es crucial en caso de que surjan problemas durante una implementación. Las estrategias que permiten revertir fácilmente los cambios ofrecen una capa adicional de seguridad y facilitan la corrección de errores.

Velocidad de Implementación: En un entorno competitivo, la velocidad de implementación es clave para mantenerse ágil y responder rápidamente a las necesidades del mercado. Estrategias de despliegue que permiten implementaciones rápidas y eficientes son esenciales para la innovación continua.

Impacto en la Experiencia del Usuario: La experiencia del usuario juega un papel crucial en la retención de clientes y la lealtad a la marca. Las estrategias de despliegue que minimizan el impacto en la experiencia del usuario aseguran que los cambios se realicen de manera transparente y sin interrupciones para los usuarios finales.

Necesidad de Infraestructura Adicional: La infraestructura adicional puede aumentar los costos operativos y la complejidad de gestión. Es importante considerar la necesidad de recursos adicionales al seleccionar una estrategia de despliegue.

Tolerancia a Fallos: La capacidad de una estrategia de despliegue para manejar fallos de manera efectiva puede marcar la diferencia entre una implementación exitosa y un desastre. Estrategias que son resistentes a fallos y tienen mecanismos de recuperación sólidos son preferibles en entornos

críticos.

Complejidad de Gestión del Estado: La gestión del estado del sistema durante las actualizaciones es crucial para garantizar la coherencia y la integridad de los datos. Estrategias de despliegue que facilitan la gestión del estado son importantes para evitar inconsistencias y problemas de integridad de datos.

Escalabilidad: A medida que un sistema crece y evoluciona, es importante que las estrategias de despliegue sean capaces de escalar para satisfacer las demandas cambiantes. Estrategias que son altamente escalables pueden adaptarse fácilmente a aumentos en la carga de trabajo y a cambios en las necesidades del negocio.

La elección de la estrategia adecuada dependerá de factores expuestos previamente, así como los requisitos específicos del negocio, el nivel de riesgo tolerado y la capacidad de la organización para gestionar y adaptarse a los cambios. A continuación, se describe una tabla con el detalle de las ventajas y desventajas sobre cada uno de los factores descritos previamente en comparación con las estrategias de despliegue detalladas al inicio de la presente sección de este informe:

Factor	Descripción	Ventajas	Desventajas
Impacto en Disponibilidad	Impacto que tiene la estrategia en la disponibilidad del servicio durante el despliegue de nuevas versiones.	Blue-Green y Rolling Deployment: Minimizan el impacto en la disponibilidad.	Canary Deployment y A/B Testing: Pueden afectar temporalmente la disponibilidad para un subconjunto de usuarios.
Riesgo de Tiempo de Inactividad	Riesgo asociado al tiempo de inactividad del servicio durante el despliegue de nuevas versiones.	Deployment Rings y Rollback Automático: Reducen el riesgo de tiempo de inactividad.	Blue-Green Deployment: Puede aumentar el riesgo de tiempo de inactividad si la conmutación entre versiones no se realiza correctamente.
Complejidad de Implementación	Nivel de complejidad requerido para implementar y mantener la estrategia de despliegue.	Feature Toggles y Deployment Rings: Ofrecen una implementación relativamente simple.	Shadow Deployment y A/B Testing: Pueden requerir una infraestructura de monitoreo y registro más compleja.
Capacidad de Revertir Cambios	Facilidad con la que se pueden revertir los cambios en caso de fallos o problemas durante el despliegue.	Rollback Automático: Permite revertir automáticamente a una versión anterior.	Canary Deployment y A/B Testing: Pueden no tener un mecanismo directo para revertir cambios.
Velocidad de Implementación	Velocidad con la que se pueden implementar nuevas versiones de la aplicación en producción.	Deployment Rings y Rolling Deployment: Permiten implementaciones rápidas y continuas.	Blue-Green Deployment: Puede ser más lento debido a la necesidad de mantener dos entornos activos.
Impacto en Experiencia del Usuario	Impacto que tiene la estrategia en la experiencia del usuario final durante el despliegue de nuevas versiones.	Feature Toggles: Permite activar y desactivar características sin impactar a todos los usuarios.	Canary Deployment y A/B Testing: Pueden afectar la experiencia del usuario para un subconjunto de usuarios.

Necesidad de Infraestructura Adicional	Requisitos adicionales de infraestructura necesarios para implementar la estrategia de despliegue.	Blue-Green y Shadow Deployment: No requieren infraestructura adicional.	Canary Deployment y A/B Testing: Pueden requerir duplicación de recursos.
Tolerancia a Fallos	Capacidad de la estrategia para tolerar fallos durante el despliegue y recuperarse de ellos.	Rollback Automático: Proporciona una forma de recuperarse rápidamente de fallos.	Blue-Green Deployment: Puede no tener un mecanismo automático para revertir cambios en caso de fallo.
Complejidad de Gestión del Estado	Nivel de complejidad asociado con la gestión del estado de la aplicación durante el despliegue.	Rolling Deployment: Facilita la gestión del estado entre versiones.	Feature Toggles: Puede introducir complejidad en el código y la gestión del estado de las características.
Escalabilidad	Capacidad de la estrategia para escalar de manera efectiva y eficiente en entornos de producción.	Deployment Rings y Rolling Deployment: Son escalables y adecuados para implementaciones continuas y de gran escala.	Blue-Green Deployment: Puede requerir más recursos debido a la necesidad de mantener dos entornos activos.

Estrategias de Despliegues para Arquitecturas Monolíticas y Resilientes

Las Estrategias de Despliegues en arquitecturas monolíticas y resilientes son enfoques y técnicas diseñadas para realizar la actualización de las aplicaciones manteniendo la disponibilidad del servicio. A continuación, se analizan los aspectos a considerar para ambas arquitecturas.

Al haber detallado los principales factores para la definición de uso sobre las estrategias de despliegue es importante realizar una comparación del uso de estas en relación con los dos tipos de arquitecturas analizadas como son las monolíticas y resilientes.

Factor	Arquitectura Monolítica	Arquitectura Resiliente
Impacto en Disponibilidad	Mayor riesgo de afectación total si falla una parte	Menor impacto debido a que los servicios son independientes
Riesgo de Tiempo de Inactividad	Alto, debido a la naturaleza centralizada de la aplicación	Bajo, ya que los fallos pueden ser aislados a servicios individuales
Complejidad de Implementación	Menor, debido a una estructura unificada	Mayor, requiere coordinación y manejo de múltiples servicios
Capacidad de Revertir Cambios	Baja, revertir cambios puede afectar toda la aplicación	Alta, se puede revertir cambios en servicios individuales sin afectar el resto
Velocidad de Implementación	Inicialmente más rápida debido a la simplicidad	Más lenta al inicio, pero puede acelerarse con experiencia y herramientas adecuadas
Impacto en Experiencia del Usuario	Potencialmente mayor si hay fallos, ya que afectan toda la aplicación	Menor, ya que los fallos en servicios individuales pueden ser manejados con degradación graciosa
Necesidad de Infraestructura Adicional	Menor, generalmente se requiere un entorno más simple	Mayor, se necesita infraestructura para orquestación, monitoreo y gestión de múltiples servicios

Tolerancia a Fallos	Baja, un fallo puede afectar toda la aplicación	Alta, los servicios independientes pueden fallar sin afectar la totalidad del sistema
Complejidad de Gestión del Estado	Menor, ya que el estado suele estar centralizado	Mayor, ya que se requiere manejo distribuido del estado y consistencia entre servicios
Escalabilidad	Limitada, escalabilidad vertical es más común	Alta, se puede escalar horizontalmente cada servicio de forma independiente

Las arquitecturas monolíticas son menos complejas en el despliegue inicial porque el sistema se despliega como un solo artefacto. Sin embargo, la gestión de dependencias internas es más complicada. En contraste, las arquitecturas de microservicios, aunque requieren herramientas de orquestación avanzadas, permiten una gestión más granular y flexible de los componentes.

Las arquitecturas de microservicios ofrecen una flexibilidad superior en términos de despliegue y actualización de componentes individuales sin afectar el resto del sistema. Esto facilita la implementación de despliegues continuos (CD) y permite actualizaciones más frecuentes, en comparación con las arquitecturas monolíticas, donde las actualizaciones son menos frecuentes y requieren el despliegue completo de la aplicación.

La arquitectura de microservicios tiene una ventaja significativa en términos de mitigación de riesgos y recuperación. Un fallo en un microservicio específico no compromete el sistema completo y permite un rollback más sencillo y menos riesgoso. En cambio, en una arquitectura monolítica, un fallo puede afectar toda la aplicación y revertir a una versión anterior es más complejo y riesgoso.

Las arquitecturas de microservicios proporcionan una escalabilidad más eficiente y granular, permitiendo escalar solo los servicios que lo necesitan, optimizando así los recursos y costos. Además, aunque el monitoreo de microservicios requiere una infraestructura más avanzada y herramientas de observabilidad, ofrece una visión detallada y precisa del desempeño y problemas del sistema, a diferencia de la arquitectura monolítica, donde el monitoreo es más sencillo, pero identificar el origen de los problemas es más difícil.

Desafíos y Ventajas en Estrategias de Despliegue en Arquitecturas Monolíticas y Resilientes

Una vez analizados los factores a considerar en cada una de las estrategias de despliegue y su comparación con las arquitecturas resilientes y monolíticas, es esencial realizar una comparación detallada de los desafíos y ventajas asociados con la adopción de cada una de ellas. Este análisis permitirá tomar decisiones informadas sobre qué enfoque adoptar, considerando sus necesidades específicas, recursos disponibles y consideraciones particulares de cada despliegue a ser realizado:

Estrategia	Arquitecturas Monolíticas	Arquitecturas Resilientes
Blue-Green Deployment	Desafíos: Requiere duplicar toda la infraestructura, lo que puede ser costoso y complejo de gestionar. Cambiar el tráfico entre entornos también puede ser arriesgado si algo sale mal. Ventajas: Permite una transición sin tiempo de inactividad, lo que es crucial para aplicaciones críticas.	Desafíos: Necesita coordinación entre múltiples servicios para asegurar que todos los componentes funcionan correctamente antes de cambiar el tráfico. Ventajas: Facilita despliegues sin interrupciones y pruebas de integración entre servicios antes de la transición completa.
Rolling Deployment	Desafíos: La actualización gradual puede ser complicada, ya que cada instancia debe ser manejada cuidadosamente para evitar inconsistencias. Ventajas: Reduce el riesgo de un despliegue fallido afectando a todos los usuarios al mismo tiempo.	Desafíos: Requiere una gestión precisa del estado y versiones de los servicios. Ventajas: Permite actualizaciones continuas con mínimo impacto en el usuario, ya que solo partes del sistema se actualizan progresivamente.
A/B Testing	Desafíos: Difícil de implementar debido a la necesidad de manejar múltiples versiones de la aplicación simultáneamente. Ventajas: Proporciona datos valiosos sobre el comportamiento del usuario con diferentes versiones.	Desafíos: Requiere coordinación entre servicios para manejar la asignación de usuarios a diferentes versiones. Ventajas: Permite la experimentación continua y el despliegue de nuevas características de manera segura, con impactos aislados a servicios específicos.
Shadow Deployment	Desafíos: Complejo debido a la necesidad de replicar tráfico real sin impactar la aplicación en producción. Ventajas: Permite probar nuevas versiones con tráfico real sin afectar a los usuarios.	Desafíos: Necesita infraestructura para duplicar y aislar el tráfico, lo cual puede ser costoso. Ventajas: Facilita la validación de servicios en un entorno de producción real, asegurando que funcionen correctamente antes del despliegue completo.
Feature Toggles	Desafíos: Puede complicar el código base con múltiples condiciones y versiones de características. Ventajas: Permite activar o desactivar características sin necesidad de redeploy.	Desafíos: Requiere una gestión cuidadosa de la configuración para asegurar consistencia. Ventajas: Facilita la entrega continua, permitiendo a los desarrolladores lanzar características y desactivarlas según sea necesario.

Deployment Rings	Desafíos: Puede ser complicado dividir a los usuarios en anillos de despliegue debido a la naturaleza integral de la aplicación. Ventajas: Permite una implementación gradual, mitigando riesgos.	Desafíos: Requiere gestión precisa de versiones y segmentación de usuarios. Ventajas: Muy eficaz para despliegues graduales y controlados, permitiendo validar cada anillo antes de proceder al siguiente.
Rollback Automático	Desafíos: Complejo debido a la necesidad de revertir toda la aplicación, lo cual puede llevar tiempo y ser riesgoso. Ventajas: Minimiza el tiempo de inactividad en caso de fallos graves.	Desafíos: Requiere monitoreo constante y mecanismos automatizados para detectar fallos. Ventajas: Muy efectivo para revertir rápidamente servicios individuales sin afectar al sistema completo, reduciendo el impacto de errores.

Comparativa de las Estrategias de Despliegue vs Factores a Considerar

Comparación entre las estrategias de despliegue analizadas en el presente informe técnico. Es importante tener en cuenta que la adopción o adecuación de una estrategia particular de despliegue dependerá de los requisitos específicos del proyecto.

Estrategia de Despliegue	Descripción	Impacto en Disponibilidad	Riesgo de Tiempo de Inactividad	Complejidad de Implementación	Capacidad de Revertir Cambios	Velocidad de Implementación	Impacto en Experiencia del Usuario	Necesidad de Infraestructura Adicional	Tolerancia a Fallos	Complejidad de Gestión del Estado	Escalabilidad
Blue-Green Deployment	Dos entornos activos (azul y verde) para conmutar tráfico.	Minimiza	Aumenta	Alta	Media	Media	Media	Alta	Media	Media	Alta
Canary Deployment	Lanzamiento gradual a subconjunto de usuarios o servidores.	Media	Media	Media	Baja	Alta	Media	Baja	Media	Media	Alta
Rolling Deployment	Despliegue gradual con actualización progresiva.	Media	Media	Media	Media	Alta	Media	Baja	Media	Alta	Alta
A/B Testing	Lanzamiento simultáneo de dos versiones para comparar.	Media	Media	Alta	Baja	Media	Media	Baja	Media	Media	Media
Shadow Deployment	Implementación paralela sin tráfico de producción.	Media	Media	Media	Media	Media	Media	Baja	Media	Media	Media
Feature Toggles	Habilitar/deshabilitar funcionalidades en tiempo de ejecución.	Media	Media	Alta	Media	Alta	Media	Baja	Media	Alta	Media
Deployment Rings	Implementación escalonada en anillos de servidores.	Media	Media	Media	Media	Alta	Media	Media	Media	Media	Alta
Rollback Automático	Revertir automáticamente a una versión anterior en caso de fallo.	Media	Media	Media	Alta	Media	Media	Baja	Alta	Media	Media

En base al estudio realizado sobre las estrategias de despliegue y los factores que pueden afectar a cada una de ellas se pudo resumir que las estrategias de despliegue “Blue Green Deployment”, “Canary Deployment” y “Rolling Deployment” son ser las estrategias más equilibradas, en el enfoque de aplicaciones basadas en arquitecturas resilientes, debido a su capacidad de minimizar riesgos y su facilidad para revertir cambios rápidamente.

4. Análisis de Casos de Uso y Mejores Prácticas para Estrategias de Despliegue en Arquitecturas Resilientes

Blue-Green Deployment:

Casos de Uso: Ideal para aplicaciones críticas que requieren cambios frecuentes sin tiempo de inactividad.

Útil cuando se necesita realizar pruebas en un entorno idéntico antes de la implementación completa.

Mejores Prácticas:

- Mantener dos entornos de producción idénticos.
- Redirigir el tráfico de producción al nuevo entorno una vez que se ha verificado su estabilidad.
- Automatizar el proceso de conmutación entre entornos para minimizar errores humanos.
- Monitorear continuamente la salud y el rendimiento de ambos entornos.

Canary Deployment:

Casos de Uso: Útil cuando se desea probar una nueva funcionalidad con un grupo limitado de usuarios antes de lanzarla a todos.

Mejores Prácticas:

- Identificar un grupo de usuarios canarios representativo para probar la nueva función.
- Implementar la nueva función en un entorno de producción en paralelo al entorno principal.
- Aumentar gradualmente el número de usuarios que tienen acceso a la nueva función.
- Recopilar comentarios y métricas de los usuarios canarios para evaluar la efectividad de la nueva función.

Rolling Deployment:

Casos de Uso: Adecuado para aplicaciones que deben actualizarse con frecuencia sin tiempo de inactividad significativo.

Mejores Prácticas:

- Actualizar las instancias de la aplicación una por una para minimizar el impacto en el servicio.
- Implementar un mecanismo de rollback automático en caso de que una actualización cause problemas.
- Utilizar herramientas de monitoreo para verificar la salud de las instancias actualizadas.
- Coordinar cuidadosamente las actualizaciones para evitar sobrecargar el sistema.

A/B Testing:

Casos de Uso: Útil para evaluar el impacto de cambios en la interfaz de usuario, características o algoritmos.

Mejores Prácticas:

- Definir claramente el objetivo del experimento y las métricas a medir.
- Dividir aleatoriamente los usuarios en grupos de control y experimento.
- Implementar la variación A en un grupo y la variación B en otro.
- Recopilar y analizar datos para determinar qué variación es más efectiva.

Shadow Deployment:

Casos de Uso: Útil para probar cambios de forma segura sin afectar a los usuarios en producción.

Mejores Prácticas:

- Enviar copias de las solicitudes de producción a un entorno de prueba en paralelo.
- No cambiar el comportamiento del sistema basado en las respuestas del entorno de prueba.
- Utilizar herramientas de monitoreo para comparar el comportamiento del entorno de prueba con el de producción.
- Implementar alertas para detectar cualquier discrepancia entre los dos entornos.

Feature Toggles:

Casos de Uso: Útil para habilitar o deshabilitar funcionalidades de forma dinámica sin necesidad de una nueva implementación.

Mejores Prácticas:

- Implementar controles de características flexibles que puedan activarse o desactivarse fácilmente.
- Monitorear el rendimiento y la estabilidad de las características habilitadas.
- Establecer un plan claro para eliminar las características obsoletas y sus correspondientes toggles.
- Mantener un registro de cambios en los toggles para rastrear quién y cuándo se activaron o desactivaron.

Deployment Rings:

Casos de Uso: Útil para implementar cambios en etapas graduales en un entorno de producción.

Mejores Prácticas:

- Dividir los usuarios o instancias de la aplicación en anillos de despliegue basados en criterios como la ubicación geográfica o el tipo de usuario.
- Implementar cambios en un anillo de despliegue a la vez para controlar el impacto y facilitar la detección de problemas.
- Monitorear las métricas clave en cada anillo de despliegue para identificar cualquier problema temprano.
- Ajustar la velocidad y el alcance del despliegue en función de los resultados obtenidos en cada anillo.

Rollback Automático:

Casos de Uso: Útil para mitigar rápidamente los problemas que surgen después de una implementación.

Mejores Prácticas:

- Implementar una política clara para determinar cuándo activar el rollback automático, como la detección de errores críticos o un deterioro significativo del rendimiento.
- Automatizar el proceso de rollback para minimizar el tiempo de inactividad y reducir el riesgo de errores humanos.
- Registrar todos los eventos de rollback y realizar análisis posteriores para identificar posibles mejoras en el proceso.
- Realizar pruebas exhaustivas del rollback automático para garantizar su eficacia y fiabilidad en situaciones de emergencia.

5. Herramientas para las fases Monitoreo y Operación

Las herramientas de monitoreo y bservabilidad son componentes críticos para gestionar cualquier sistema distribuido, y contenedores no es la excepción, la complejidad inherente de un clúster de Kubernetes puede dificultar la identificación y resolución de problemas sin las herramientas adecuadas.

El monitoreo en Kubernetes implica la recopilación, almacenamiento y análisis de métricas sobre el estado de los diversos componentes del clúster y las aplicaciones que se ejecutan en él. Esto incluye:

Métricas de Infraestructura: Información sobre el uso de CPU, memoria, almacenamiento y red de los nodos y pods.

Métricas de Aplicaciones: Datos sobre el rendimiento y la salud de las aplicaciones desplegadas, como latencia de respuesta y tasas de error.

Eventos y Logs: Registro de eventos importantes y logs generados por los componentes del clúster y las aplicaciones.

En el presente informe se ha mencionado lo correspondiente al monitoreo de los contenedores, sin embargo, con la adopción de estos y con el uso particular de kubernetes es importante enfatizar que el monitoreo tradicional no satisface todas las necesidades de los equipos de TI, razón por la cual es indispensable incorporar la observabilidad.

La Observabilidad va más allá del monitoreo tradicional al proporcionar una visión más profunda y holística del sistema. Los tres pilares de la observabilidad son:

Logs: Registros detallados de eventos y actividades dentro del sistema.

Métricas: Datos cuantitativos sobre el comportamiento y el rendimiento del sistema.

Tracing: Seguimiento de las solicitudes a través de los diferentes servicios y componentes, facilitando la identificación de cuellos de botella y la detección de fallos en aplicaciones distribuidas.

Se han definido los puntos principales a ser abordados con la observabilidad de aplicaciones, la detección proactiva de problemas que permite Identificar problemas antes de que afecten a los usuarios finales, la resolución rápida de incidentes que facilitará el diagnóstico y la resolución rápida de fallos de las aplicaciones, la optimización del rendimiento lo que permitirá realizar ajustes basados en datos reales para mejorar el rendimiento de los aplicativos, finalmente la seguridad y cumplimiento, donde se realizará el monitoreo a las actividades sospechosas que permitirán identificar que el sistema cumple con las normativas de seguridad.

Con el cambio de la arquitectura tradicional hacia las resilientes, cambian las herramientas con las que se realiza el monitoreo de las aplicaciones e infraestructura. A continuación se detallan algunas de las herramientas más comunes utilizadas para este tipo de arquitecturas:

Prometheus: Es una de las herramientas de monitoreo más populares para contenedores. Es una solución de código abierto que recopila métricas en tiempo real, proporciona almacenamiento en series temporales y cuenta con un potente lenguaje de consulta (PromQL) para analizar y visualizar los datos. Prometheus se integra bien con Kubernetes, y hay muchos exportadores disponibles para diferentes servicios y aplicaciones

Grafana: Complementa a Prometheus proporcionando un panel de control flexible y potente para la visualización de datos. Puedes crear paneles personalizados y alertas basadas en las métricas recopiladas por Prometheus u otras fuentes de datos.

Elasticsearch, Fluentd, Kibana (EFK Stack): El ELK Stack es una solución de código abierto para la búsqueda, análisis y visualización de datos de registro en tiempo real. Elasticsearch es el motor de búsqueda y análisis, Logstash recopila y procesa los datos de registro, y Kibana proporciona la interfaz de usuario para la visualización.

Datadog: Es una plataforma de monitoreo y análisis que proporciona una visión integral del rendimiento de tus aplicaciones, infraestructura y servicios. Ofrece integración con Kubernetes y Docker, y proporciona paneles interactivos, alertas y trazabilidad.

New Relic: Es una plataforma de monitoreo y observabilidad que proporciona métricas detalladas sobre el rendimiento de las aplicaciones, la infraestructura y los contenedores. Ofrece una interfaz intuitiva para el análisis y la visualización de datos, así como herramientas de alerta y diagnóstico.

Sysdig: Proporciona monitoreo de contenedores, seguridad y análisis forense. Ofrece visibilidad en tiempo real de los contenedores y las aplicaciones que se ejecutan en ellos, con capacidades de análisis de rendimiento y seguridad.

cAdvisor (Container Advisor): Es una herramienta de monitoreo de contenedores desarrollada por Google. Se integra con Kubernetes y Docker para proporcionar información sobre el uso de recursos (CPU, memoria, red y disco) por contenedor. Es una herramienta básica y ligera adecuada para obtener métricas en tiempo real.

Zabbix: Es una herramienta de monitoreo de red y aplicaciones que también puede utilizarse para monitorear contenedores. Ofrece monitoreo de rendimiento, alertas y gráficos históricos. Puede integrarse con Docker y Kubernetes a través de complementos y scripts personalizados.

Nagios: Es una herramienta de monitoreo de infraestructura que puede adaptarse para el monitoreo de contenedores mediante complementos. Proporciona monitoreo de la disponibilidad, el rendimiento y la capacidad, y permite configurar alertas personalizadas.

Jaeger: Es una herramienta de trazabilidad de código abierto diseñada para monitorear y solucionar problemas en sistemas distribuido.

Zipkin: Es otra herramienta de trazabilidad de código abierto ermite rastrear solicitudes a través de diferentes servicios y microservicios, identificando dónde se están produciendo latencias o fallos.

A continuación, se realiza una tabla comparativa de las herramientas descritas previamente haciendo énfasis en la integración con Ansible como medio para ejecutar remediaciones automáticas sobre eventos en la infraestructura.

Herramientas	Tipo	Características Principales	Integraciones Clave	Código Abierto/Comercial	Integrable con Ansible
Prometheus	Monitoreo	Recopilación de métricas, almacenamiento en series temporales, PromQL	Kubernetes, Docker, Grafana	Código Abierto	Sí
Grafana	Visualización	Paneles personalizables, alertas, integración con múltiples fuentes de datos	Prometheus, Elasticsearch, InfluxDB, Loki	Código Abierto	Sí
Kubernetes Dashboard	Monitoreo	Interfaz web para Kubernetes, monitoreo de recursos del clúster, gestión de aplicaciones	Kubernetes	Código Abierto	Sí (indirectamente)
ELK Stack	Log Management	Elasticsearch para búsqueda, Logstash para procesamiento, Kibana para visualización	Kubernetes, Docker, Beats	Código Abierto	Sí
Datadog	Monitoreo/Observabilidad	Monitoreo de aplicaciones, infraestructura, servicios, trazabilidad, paneles interactivos	Kubernetes, Docker, AWS, Azure, GCP	Comercial	Sí

New Relic	Monitoreo/Observabilidad	Monitoreo de rendimiento de aplicaciones, infraestructura, trazabilidad, alertas	Kubernetes, Docker, AWS, Azure, GCP	Comercial	Sí
Sysdig	Monitoreo/Seguridad	Monitoreo en tiempo real, seguridad de contenedores, análisis forense	Kubernetes, Docker, Prometheus	Comercial	Sí
cAdvisor	Monitoreo	Métricas de uso de recursos por contenedor, integración sencilla con Kubernetes y Docker	Kubernetes, Docker	Código Abierto	Sí
Zabbix	Monitoreo	Monitoreo de red y aplicaciones, alertas, gráficos históricos	Kubernetes, Docker (mediante scripts/plugins)	Código Abierto	Sí
Nagios	Monitoreo	Monitoreo de disponibilidad, rendimiento, alertas personalizadas	Kubernetes, Docker (mediante plugins)	Código Abierto	Sí
Jaeger	Trazabilidad	Trazabilidad de solicitudes, análisis de latencia, compatibilidad con OpenTracing	Kubernetes, Docker, Prometheus	Código Abierto	Sí
Zipkin	Trazabilidad	Trazabilidad de solicitudes, análisis de dependencias, compatibilidad con OpenTracing	Kubernetes, Docker	Código Abierto	Sí

Con la adopción de arquitecturas resilientes el monitoreo y la observabilidad en contenedores son esenciales para mantener la salud, el rendimiento y la seguridad para las aplicaciones desplegadas sobre esta nueva arquitectura. Implementar estas prácticas permitirá a los equipos de TI como operaciones, control de calidad y desarrollo tener una visión clara y accionable sobre el estado de los aplicativos, facilitando una gestión eficiente y proactiva.

Es importante conocer y estudiar las herramientas que se cuentan en el mercado para establecer una estrategia bien definida, que permita en lo posible mantener la estabilidad y eficiencia de los entornos de contenedores a gran escala.

Parte de la estrategia que se defina debe contemplar:

Visibilidad y Monitoreo Integral de los componentes, que debe contemplar la recopilación de métricas de recursos como CPU, memoria y uso de disco, sin dejar de lado el monitoreo del rendimiento de las aplicaciones y el tráfico de la red.

Alertas y Notificaciones, que debe permitir la definición de umbrales críticos para crear alertas tempranas sobre errores y problemas, estas deben conectarse con aplicaciones como que puedan interactuar rápidamente con los administradores.

Centralización de Logs y Rastreo (Logging y Tracing), es crucial comprender el comportamiento de las aplicaciones por medio de los logs que las mismas generan, la centralización de esta información es crucial para comprender el funcionamiento de la aplicación y los errores correlacionados que estos pueden tener con otros componentes de la infraestructura.

Visualización de métricas, la utilización de herramientas como Grafana, ELK permiten tener métricas personalizadas sobre la infraestructura gestionada facilitando la detección temprana de eventos en la infraestructura.

Escalabilidad y Rendimiento, el asegurar que las herramientas de monitoreo están preparadas para recibir el crecimiento dinámico de las aplicaciones es crucial para un correcto seguimiento del rendimiento de la infraestructura y aplicaciones.

Facilidad de Uso y Mantenimiento, indispensable para mantener una infraestructura estable el mantener una documentación clara sobre los procesos de monitoreo y observabilidad.

La integración con herramientas de automatización es indispensable para mantener una infraestructura de contenedores automatizada, en el mercado existen otras, pero la más popular es Ansible, que actualmente se utiliza en el ambiente de producción de la institución.

6. Conclusiones

Conclusiones sobre las estrategias de despliegue y su aplicabilidad en el SRI:

- Las Estrategias de Despliegues son enfoques y técnicas utilizadas para actualizar y lanzar nuevas versiones de aplicaciones, sin que se generen pérdidas al servicio.
- Cada estrategia de despliegue tiene sus propios beneficios y desafíos, y la elección de una sobre otra depende de los objetivos específicos del proyecto de desarrollo y las necesidades del equipo de DevOps.
- Las estrategias de despliegue “Blue Green Deployment”, “Canary Deployment” y “Rolling Deployment” son las más equilibradas para aplicaciones basadas en arquitecturas resilientes en base al análisis realizado y, son las más adecuadas para el uso con aplicaciones en arquitecturas resilientes.
- La elección de la estrategia de despliegue debe ser una decisión informada basada en una evaluación detallada de las características de la aplicación, los requisitos del negocio, y las capacidades de la infraestructura.
- La estrategia de despliegue debe minimizar los riesgos operacionales, y deben maximizar la disponibilidad del aplicativo

7. Recomendaciones

- Realizar una evaluación de la aplicación y su arquitectura, considerando factores como la criticidad del servicio, la tolerancia al fallo, y la necesidad de disponibilidad para definir las estrategias de despliegue a usar.
- Considerar las estrategias de despliegue Blue Green Deployment, Canary Deployment y Rolling Deployment que, en base al análisis han demostrado ser las más equilibradas para aplicaciones resilientes.

- Capacitar a los equipos de TI sobre las estrategias de despliegue existentes y las herramientas de automatización para implementar los despliegues de aplicaciones de manera eficiente.
- Implementar herramientas de observabilidad como las mencionadas en el presente informe para los ambientes de desarrollo, pruebas, producción con la finalidad de tener las mismas métricas y análisis en todos los ambientes.
- La estrategia de despliegue debe estar alineada con los objetivos estratégicos de la organización. Esto incluye considerar la escalabilidad, los costos, la seguridad y el cumplimiento con las normativas vigentes.

8. Glosario

- CI/CD.- Continuous Integration / Continuous Deployment; Integración Continua / Despliegue Continuo
- DevOps.- Acrónimo inglés de development (Desarrollo) y operations (operaciones)

9. Bibliografía

Understand modern app deployment strategies with Oracle Cloud Infrastructure DevOps. (2022, 20 mayo). Oracle Help Center. <https://docs.oracle.com/es/solutions/mod-app-deploy-strategies-oci/index.html>

Understand deployment strategies. (2022, 10 mayo). Oracle Help Center. <https://docs.oracle.com/es/solutions/plan-mad-strats-devops/understand-depoyment-architectures.html>

Determine your deployment strategy. (2022b, julio 15). Oracle Help Center. <https://docs.oracle.com/en/solutions/design-dr/determine-your-deployment-strategy1.html>

Best practices for operating cloud deployments efficiently. (s. f.). Oracle Help Center. <https://docs.oracle.com/en/solutions/oci-best-practices-operations/plan-your-deployment-strategy1.html>

Deployment Strategies - Deployments | Developer Guide | OpenShift Container Platform 3.11. (s. f.). OpenShift. https://docs.openshift.com/container-platform/3.11/dev_guide/deployments/deployment_strategies.html

Using deployment strategies OpenShift Container Platform 4.5 | Red Hat Customer Portal. (s. f.). Red Hat Customer Portal. https://access.redhat.com/documentation/zh-cn/openshift_container_platform/4.5/html/applications/deployment-strategies#:~:text=A%20deployment%20strategy%20is%20a%20way%20to%20change%20or%20upgrade%20an%20application.

Flow Modeling: How Work Moves Through the Enterprise - Plutora. (s. f.). <https://www.plutora.com/blog/deployment-strategies-6-explained-in-depth>

Haddad, R. (2024, 15 mayo). Rollout and Deployment Strategies: Definition, Types and the Role of Feature Flags in Your Deployment Process. Abtasty. <https://www.abtasty.com/blog/deployment-strategies/>

Understand modern app deployment strategies with Oracle Cloud Infrastructure DevOps. (2022b, mayo 20). Oracle Help Center. <https://docs.oracle.com/en/solutions/mod-app-deploy-strategies-oci/index.html>

La integración y la distribución continuas (CI/CD). (s. f.). <https://www.redhat.com/es/topics/devops/what-is-ci-cd>

Espejel, A. (2022, 18 mayo). Estrategias para desplegar con éxito en producción. Paradigma Digital. <https://www.paradigmadigital.com/dev/estrategias-desplegar-exito-produccion/>

¿Qué es la implementación azul-verde? (s. f.). <https://www.redhat.com/es/topics/devops/what-is-blue-green-deployment>

Red Hat presenta una estrategia completa de Platform-as-a-Service para Cloud Computing. (s. f.). <https://www.redhat.com/es/about/press-releases/4733>

Mallón, X. (2024, 24 abril). Glosario: 4 estrategias de despliegue. KeepCoding Bootcamps. <https://keepcoding.io/blog/glosario-estrategias-de-despliegue/>

Espejel, A. (2022b, mayo 18). Estrategias para desplegar con éxito en producción. Paradigma Digital. <https://www.paradigmadigital.com/dev/estrategias-desplegar-exito-produccion/>

Livens, J. (2023, 12 diciembre). Observability vs. monitoring: What's the difference? Dynatrace News. <https://www.dynatrace.com/news/blog/observability-vs-monitoring/#:~:text=Monitoring%20and%20observability%20provide%20a,better%20understand%20of%20observability%20vs.>

Observation vs. Monitoring: What's the Difference in the World of Cybersecurity. (s. f.). Tripwire. <https://www.tripwire.com/state-of-security/observation-vs-monitoring-whats-the-difference-in-the-world-of-cybersecurity>

Observabilidad en comparación con la supervisión: Diferencia entre los procesos basados en datos: AWS. (s. f.). Amazon Web Services, Inc. <https://aws.amazon.com/es/compare/the-difference-between-monitoring-and-observability/#:~:text=Monitoring%20is%20the%20process%20of,the%20root%20cause%20of%20issues.>

Observability vs. Monitoring: Understanding the Difference | StrongDM. (n.d.). Discover.strongdm.com. etrieved June 11, 2024, from <https://www.strongdm.com/blog/observability-vs-monitoring>

Engineer, M. T.-M. L. (2023, July 12). Monitoreo vs Observabilidad: Diferencias y Usos. Building Nubank. <https://building.nubank.com.br/es/monitoreo-vs-observabilidad-diferencias-usos/>

Observabilidad y Monitorización: Diferencias e importancia, (11/06/2024) <https://www.open3s.com/blog/2021/04/13/observabilidad-y-monitorizacion-cual-es-la-diferencia-y-por-que-es-importante-blog/>

Firmas de responsabilidad

Rubro	Cargo	Nombre	Firma
Elaborado por:	Experto Nacional de Infraestructura y Operaciones	Jorge Armijo	 Firmado electrónicamente por: JORGE LUIS ARMIJO QUITO
Revisado por:	Experto Supervisor Nacional de Infraestructura y Operaciones	Janela Menéndez	 Firmado electrónicamente por: JANELA TATIANA MENENDEZ PRIAS
Aprobado por:	Coordinador Nacional de Infraestructura y Operaciones	Miguel Sosa	 Firmado electrónicamente por: MIGUEL ALEXANDER SOSA DELGADO